Kazy Noor Sir

Exam suggestions | Compiler Design

Preparation focus: Read all four slide sets. Expect scenarios, diagrams, examples and justification.

1. Syntax-Directed Translation / SDD

- ☐ Relate **grammar, syntax-directed translation, attributes and semantic rules** to a given scenario.
- ☐ Identify **synthesized and inherited attributes** from a scenario or semantic rules. Classify an SDD as **S-attributed or L-attributed**.
- ☐ Explain the **relationship between S- and L-attributed definitions**, how each works, and their performance/evaluation differences, with examples.
- ☐ Explain **synthesized attribute evaluation order** and **bottom-up evaluation**; trace the computations.
- ☐ Build a **symbol table**. Explain how an SDD distributes attributes/type information and which attributes are synthesized or inherited.
- ☐ Given production rules and a string, draw a **parse tree or annotated parse tree**, calculate attribute values and write the relevant semantic rules.
- ☐ Explain from code/rules **when an attribute is synthesized or inherited** and how it is computed.
- ☐ Draw a **dependency graph with arrow directions**. Identify the attribute dependencies using the parse tree and explain the evaluation order.
- ☐ Justify **grammar ambiguity** and explain whether it is related to **cycles in an attribute dependency graph**.

Answer habit

For each problem: **identify the concept, show the rules or steps, draw the required diagram, and justify the result.** Use examples for comparisons and explanations.

Kazy Noor Sir

Exam suggestions | Compiler Design

Continue the checklist: intermediate code, optimization and runtime environments.

2. Intermediate Code Generation

- ☐ Explain its **relationship with SDD**, its purpose and why it supports optimization.
- ☐ Prepare **three-address code (TAC), quadruples and triples**. Choose the appropriate representation for a scenario and justify your choice.
- ☐ Prepare **basic blocks, control-flow graphs and DAGs**; show the relevant labels and dependencies.

3. Code Optimization

- ☐ Explain **why optimization is necessary**.
- ☐ Identify suitable **assembly/target-code optimization techniques** and explain the reason for each change.
- ☐ Compare given **loops or code segments**. Choose an optimization based on the scenario and justify it.
- ☐ Prepare scenario-based **multidimensional array** problems with relevant rules/functions. *The exact wording of this handwritten line is unclear.*

4. Runtime Environment

- ☐ From code/pseudocode, identify **static and dynamic memory allocation** and explain why each applies.
- ☐ Draw and trace **activation records**; identify which procedure activations are currently active and explain the execution process.
- ☐ Explain when **stack allocation** is effective. Choose the suitable runtime environment/storage allocation approach for a scenario and justify it.
- ☐ Practise **finding errors in code** and **writing a grammar** when required.
- ☐ Prepare **parameter passing**, especially **call-by-value**, with a definition, figure and example.

Source note: This is an organized checklist of the supplied handwritten suggestions, supported by the lecture materials. It is not a prediction of exact exam questions.